

AvoLaptops

Documentazione Completa delle API

Versione 1.0 — Maggio 2026

1. Introduzione

AvoLaptops è un sistema web per la gestione e la prenotazione di laptop scolastici. Permette ai docenti di riservare laptop custoditi in armadietti fisici (locker) e agli amministratori di gestire l'intero parco macchine.

L'API REST è esposta sotto il percorso base:

```
https://<host>/api/v1/
```

Tutte le risposte sono in formato JSON con header Content-Type: application/json.

1.1 Stack Tecnologico

- PHP 8.2 su Apache con mod_rewrite
- MariaDB 10.4 — database avolaptop
- PHPMailer 6.x per l'invio di email OTP
- Autenticazione JWT custom (algoritmo HS256)

1.2 Struttura delle Cartelle (API)

```
api/v1/
  index.php          - Router principale (front controller)
  .htaccess          - Riscrittura URL verso index.php
  connections/
    connection.php   - Connessione MySQLi
  access/
    login.php        - Autenticazione e generazione JWT
  checks/
    check_token.php  - Validazione JWT (middleware)
    check_admin.php  - Verifica ruolo admin
  home/
    homepage.php     - Endpoint di health-check
  resources/
    laptops/         - CRUD laptop
    lockers/         - CRUD locker
    models/          - CRUD modelli hardware
    reservations/    - CRUD prenotazioni
    users/           - CRUD utenti
    otp.php          - Verifica OTP email
```

PHPMailer/

– Libreria email (MIT)

2. Architettura e Routing

2.1 Front Controller (index.php)

Ogni richiesta viene instradata da Apache verso index.php tramite il file .htaccess:

```
Options -MultiViews
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [QSA,L]
```

Il router estrae la risorsa chiamata dall'URI con questa logica:

```
// URI esempio: /api/v1/laptops?id=laptop=3
$requestUri  = $_SERVER['REQUEST_URI'];
$uri_elements = array_slice(explode('/', $requestUri), 4);
// uri_elements[0] = 'laptops'
$called_resource = explode('?', $uri_elements[count($uri_elements)-1])[0];
```

Il routing avviene tramite un doppio switch su REQUEST_METHOD e risorsa:

```
switch ($_SERVER['REQUEST_METHOD']) {
    case 'GET':    // laptops, reservations, users, lockers, models,
homepage
    case 'POST':   // login, users, laptops, reservations, lockers,
models, otp
    case 'PATCH': // users
    case 'DELETE': // laptops, reservations, users, lockers
}
```

I metodi non supportati per una determinata risorsa restituiscono 405 Method Not Allowed; le risorse inesistenti restituiscono 404 Not Found.

2.2 Connessione al Database

Il file connections/connection.php apre una singola connessione MySQLi condivisa da tutti i file inclusi:

```
<?php
$host  = 'localhost';
$user  = 'root';
$pass  = '';
```

```
$dbname = 'avolaptop';  
$conn   = mysqli_connect($host, $user, $pass, $dbname);  
mysqli_set_charset($conn, 'utf8');
```

La connessione non usa password (ambiente di sviluppo locale). In produzione è necessario impostare credenziali sicure.

3. Autenticazione — JWT Custom

3.1 Formato del Token

Il sistema implementa JWT HS256 manualmente (senza librerie esterne). Il token è composto da tre parti codificate in Base64url separate da punti:

```
<header_b64>.<payload_b64>.<signature_b64>
```

Struttura del payload:

```
{
  "id-user": 42,
  "name": "Mario",
  "surname": "Rossi",
  "email": "mario@gmail.com",
  "password": "<sha256-hash>",
  "verified": 1,
  "admin": false,
  "start": 1716000000 // Unix timestamp di emissione
}
```

3.2 Firma e Segreto

La firma è calcolata con HMAC-SHA256 sul concatenato header.payload:

```
$secret = "ILOVEBARCELONAPLSIWANTTOGETBACK";
$signature = base64url_encode(
    hash_hmac('sha256',
        base64url_encode(json_encode($header)) . "." .
        base64url_encode(json_encode($payload)),
        $secret, true
    )
);
```

Nota di sicurezza: il segreto è hard-coded nel codice sorgente. In produzione va spostato in una variabile d'ambiente.

3.3 Scadenza

- Token standard: 86400 secondi (24 ore) dalla creazione
- Token OTP: 300 secondi (5 minuti) dalla creazione

La verifica della scadenza avviene in `check_token.php`:

```
if ($payload['start'] + 86400 <= time() && $called_resource != 'otp') {  
    $valid = false; // Token scaduto  
}  
  
if ($called_resource == 'otp' && $payload['start'] + 300 <= time()) {  
    $valid = false; // Token OTP scaduto  
}
```

3.4 Come Inviare il Token

Il token va passato come header HTTP personalizzato (non Authorization Bearer):

```
token:  
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjE2VjIjo0Mi4uLn0.abc123
```

3.5 Codici di Risposta Auth

- 498 Invalid Token — token assente, malformato o scaduto
- 401 Unauthorized — token valido ma utente non verificato via email, oppure non admin su endpoint admin

4. Schema del Database

4.1 Tabella users

Colonna	Tipo	Chiave	Note
id_user	INT AUTO_INCREMENT	PK	
name	VARCHAR(100)		Nome dell'utente
surname	VARCHAR(100)		Cognome
email	VARCHAR(150)	UNIQUE	Solo @gmail.com per la registrazione
password	CHAR(64)		Hash SHA-256 della password
role	ENUM('admin','teacher')		Default: teacher
verified	TINYINT(1)		0 = non verificato, 1 = email confermata
otp	MEDIUMINT		Codice OTP a 6 cifre
otp_time	TIMESTAMP		Aggiornato automaticamente — usato per scadenza OTP

4.2 Tabella models

Colonna	Tipo	Chiave	Note
id_model	INT AUTO_INCREMENT	PK	
brand	VARCHAR(100)		Es. Lenovo, HP, Dell
model	VARCHAR(100)		Nome del modello
cpu	VARCHAR(100)		
ram	INT		GB di RAM
storage	INT		GB di storage
os	VARCHAR(50)		Sistema operativo

4.3 Tabella lockers

Colonna	Tipo	Chiave	Note
id_locker	INT AUTO_INCREMENT	PK	
name_locker	VARCHAR(100)		Nome dell'armadietto
location	VARCHAR(200)		Posizione fisica

4.4 Tabella laptops

Colonna	Tipo	Chiave	Note
id_laptop	INT AUTO_INCREMENT	PK	
id_model	INT	FK → models	CASCADE DELETE/UPDATE
id_locker	INT	FK → lockers	CASCADE DELETE/UPDATE
status	ENUM('available','unavailable')		Aggiornato dagli eventi DB schedulati

4.5 Tabella reservations

Colonna	Tipo	Chiave	Note
id_reservation	INT	PK (composita)	
id_user	INT	PK + FK → users	CASCADE
id_laptop	INT	PK + FK → laptops	CASCADE
date	DATE		Data della prenotazione (YYYY-MM-DD)
time_start	TIME		Orario inizio (HH:MM:SS)

time_end	TIME		Orario fine (HH:MM:SS)
status	ENUM('active','completed')		Aggiornato automaticamente dagli eventi DB

4.6 Eventi schedulati del Database

Il database usa quattro eventi MySQL schedulati ogni minuto:

- ev_complete_reservations — imposta status='completed' sulle prenotazioni la cui data/ora fine è passata
- ev_laptop_available — rilascia i laptop (status='available') non più coinvolti in prenotazioni attive
- ev_laptop_unavailable — occupa i laptop (status='unavailable') per cui è attiva una prenotazione nella finestra oraria corrente
- ev_control_otp — elimina gli utenti non verificati il cui OTP è scaduto da più di 5 minuti

5. Riepilogo degli Endpoint

Metodo	URI	Auth	Admin	Descrizione
GET	/api/v1/homepage	No	No	Health-check
POST	/api/v1/login	No	No	Login e rilascio JWT
GET	/api/v1/laptops	Sì	No	Lista/filtra laptop
POST	/api/v1/laptops	Sì	Sì	Crea laptop
DELETE	/api/v1/laptops	Sì	Sì	Elimina laptop
GET	/api/v1/lockers	Sì	No	Lista/filtra locker
POST	/api/v1/lockers	Sì	Sì	Crea locker
DELETE	/api/v1/lockers	Sì	Sì	Elimina locker
GET	/api/v1/models	Sì	No	Lista/filtra modelli
POST	/api/v1/models	Sì	Sì	Crea modello
GET	/api/v1/reservations	Sì	No	Lista/filtra prenotazioni
POST	/api/v1/reservations	Sì	No	Crea prenotazione
DELETE	/api/v1/reservations	Sì	No*	Cancella prenotazione (propria o admin)
GET	/api/v1/users	Sì	Sì	Lista/filtra utenti
POST	/api/v1/users	No	No	Registrazione nuovo utente
PATCH	/api/v1/users	Sì	No	Modifica password utente
DELETE	/api/v1/users	Sì	Sì	Elimina utente
POST	/api/v1/otp	Sì*	No	Verifica OTP email (token non-verified)

* Per DELETE /reservations: l'utente normale può eliminare solo le proprie prenotazioni; l'admin può eliminare qualsiasi prenotazione.

* Per POST /otp: richiede token JWT anche se l'utente non è ancora verificato (token con verified=0 e scadenza 5 min).

6. Dettaglio degli Endpoint

6.1 GET /api/v1/homepage

Health-check dell'API. Non richiede autenticazione.

Risposta 200 OK:

```
{ "status": "200 OK" }
```

6.2 POST /api/v1/login

Autentica l'utente e restituisce un JWT con scadenza 24 ore.

Body (form-data / x-www-form-urlencoded):

Campo	Tipo	Obbligatorio	Descrizione
email	string	Sì	Email dell'utente
password	string	Sì	Password in chiaro (viene hashata SHA-256 lato server)

Risposta 200 OK (login riuscito):

```
{
  "status": "200 OK",
  "login_status": "Successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZC11..."
}
```

Risposta 401 — credenziali errate o account non verificato:

```
{ "status": "401 Unauthorized", "login_status": "Not valid credentials"
}
```

6.3 POST /api/v1/users — Registrazione

Crea un nuovo utente con ruolo teacher. Invia un OTP via email SMTP (Gmail) per la verifica. Non richiede autenticazione previa.

Accetta SOLO indirizzi @gmail.com. Dopo la registrazione, l'account viene eliminato automaticamente dal database se non verificato entro 5 minuti.

Body:

Campo	Tipo	Obbligatorio	Descrizione
name	string	Sì	Nome
surname	string	Sì	Cognome
email	string	Sì	Deve essere un indirizzo @gmail.com
password	string	Sì	Password in chiaro

Risposta 201 Created (successo):

```
{
  "status": "201 Created",
  "message": "Verify the account with the sent OTP on your email.",
  "token": "<jwt_non_verificato>"
}
```

Il token restituito ha verified=0 e serve solo per il successivo endpoint POST /otp. Ha una scadenza di 5 minuti.

Risposta 406 — utente già esistente:

```
{ "status": "406 Not Acceptable", "message": "User exists, if you did
not register wait 5 minutes" }
```

Dettaglio codice — invio OTP con PHPMailer:

```
$mail = new PHPMailer(true);
$mail->isSMTP();
$mail->Host      = 'smtp.gmail.com';
$mail->SMTPAuth  = true;
$mail->Username  = 'example@gmail.com';
$mail->Password  = 'sigma';
$mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
$mail->Port      = 587;
$mail->Subject   = 'Verifica la mail';
$mail->Body      = 'Il codice OTP per la verifica della mail è: ' .
$otp_code;
$mail->send();
```

6.4 POST /api/v1/otp — Verifica OTP

Verifica il codice OTP ricevuto via email e imposta verified=1 sul profilo utente. Richiede il JWT emesso dalla registrazione (token non-verified, valido 5 min).

Body:

Campo	Tipo	Obbligatorio	Descrizione
otp	integer	Sì	Codice OTP a 6 cifre ricevuto via email

Risposta 200 OK:

```
{ "status": "200 OK", "message": "OTP verified successfully" }
```

Risposta 406 — OTP errato o scaduto:

```
{ "status": "406 Not Acceptable", "message": "OTP not valid" }
```

6.5 PATCH /api/v1/users — Cambio Password

Aggiorna la password dell'utente autenticato. Non richiede l'admin. La nuova email target è ricavata dal token JWT.

Body (i dati vengono letti da php://input come query string):

Campo	Tipo	Obbligatorio	Descrizione
password	string	Sì	Nuova password in chiaro (viene hashata SHA-256 lato server)

Risposta 200 OK: payload standard senza campi aggiuntivi.

Dettaglio codice — lettura body PATCH:

```
parse_str(file_get_contents('php://input'), $_PATCH);  
$hash_pass = hash('sha256', $_PATCH['password']);  
$query = 'UPDATE users SET password = ? WHERE email = ?';
```

6.6 GET /api/v1/users — Lista Utenti (Admin)

Restituisce tutti gli utenti o filtra tramite query string. Richiede autenticazione e ruolo admin.

Query string: qualsiasi campo della tabella users, con underscore sostituito da trattino.

Esempio:

```
GET /api/v1/users?role=teacher  
GET /api/v1/users?id-user=5
```

Risposta 200:

```
{
  "status": "200 OK",
  "users": [
    { "id_user": 5, "name": "Mario", "surname": "Rossi", "email":
      "mario@gmail.com",
      "password": "<sha256>", "role": "teacher", "verified": 1, "otp":
      null, "otp_time": null }
  ]
}
```

6.7 DELETE /api/v1/users — Elimina Utente (Admin)

Elimina un utente per ID. Richiede admin. Il body va inviato come application/x-www-form-urlencoded.

Body (letto da php://input):

Campo	Tipo	Obbligatorio	Descrizione
id-user	integer	Sì	ID dell'utente da eliminare

Risposta 200 OK senza payload aggiuntivo.

6.8 GET /api/v1/laptops — Lista Laptop

Restituisce i laptop con dati del modello e del locker aggregati. Richiede autenticazione.

Filtri disponibili (query string):

- id-laptop — ID specifico del laptop
- status — available | unavailable
- id-model — filtra per modello
- id-locker — filtra per locker
- brand, model, cpu, ram, storage, os — attributi del modello

URI speciale — laptop di un locker specifico (path parameter):

```
GET /api/v1/lockers/3/laptops
```

Risposta 200:

```
{
  "status": "200 OK",
  "laptops": [
    {
      "id_laptop": 7,
      "status": "available",
      "model": { "id_model": 2, "brand": "Lenovo", "model": "ThinkPad",
"cpu": "i5", "ram": 8, "storage": 256, "os": "Windows 11" },
      "locker": { "id_locker": 1, "name_locker": "Locker A", "location":
"Piano 2" }
    }
  ]
}
```

Il codice costruisce la risposta aggregata iterando sulle colonne del result set con contatori:

```
// counter 0-1: id_laptop, status -> laptop root
// counter 2-7: colonne modello -> laptop['model']
// counter 8+: colonne locker -> laptop['locker']
```

6.9 POST /api/v1/laptops — Crea Laptop (Admin)

Body:

Campo	Tipo	Obbligatorio	Descrizione
id-model	integer	Sì	ID del modello hardware

id-locker	integer	Sì	ID del locker in cui è custodito
-----------	---------	----	----------------------------------

Risposta 201 Created.

6.10 DELETE /api/v1/laptops — Elimina Laptop (Admin)

Body:

Campo	Tipo	Obbligatorio	Descrizione
id-laptop	integer	Sì	ID del laptop da eliminare

Risposta 200 OK senza payload aggiuntivo.

6.11 GET /api/v1/lockers — Lista Locker

Restituisce tutti i locker o filtra per attributo. Richiede autenticazione.

Filtri: id-locker, name-locker, location.

Risposta 200:

```
{
  "status": "200 OK",
  "lockers": [
    { "id_locker": 1, "name_locker": "Locker A", "location": "Piano 2" }
  ]
}
```

6.12 POST /api/v1/lockers — Crea Locker (Admin)

Body:

Campo	Tipo	Obbligatorio	Descrizione
name	string	Sì	Nome del locker
location	string	Sì	Posizione fisica

Risposta 201 Created.

6.13 DELETE /api/v1/lockers — Elimina Locker (Admin)

Body:

Campo	Tipo	Obbligatorio	Descrizione
id-locker	integer	Sì	ID del locker da eliminare

Risposta 200 OK.

6.14 GET /api/v1/models — Lista Modelli

Restituisce tutti i modelli hardware o filtra per attributo. Richiede autenticazione.

Filtri: id-model, brand, model, cpu, ram, storage, os.

Risposta 200:

```
{
  "status": "200 OK",
  "models": [
    { "id_model": 1, "brand": "HP", "model": "EliteBook", "cpu": "i7",
      "ram": 16, "storage": 512, "os": "Windows 11" }
  ]
}
```

6.15 POST /api/v1/models — Crea Modello (Admin)

Body:

Campo	Tipo	Obbligatorio	Descrizione
brand	string	Sì	Produttore
model	string	Sì	Nome modello
cpu	string	Sì	Processore
ram	integer	Sì	RAM in GB
storage	integer	Sì	Storage in GB
os	string	Sì	Sistema operativo

Risposta 201 Created.

6.16 GET /api/v1/reservations — Lista Prenotazioni

Restituisce le prenotazioni con dati aggregati su utente, laptop, modello e locker. Richiede autenticazione.

Filtri: id-reservation, date, time-start, time-end, status, id-user, id-laptop, brand, model...

Risposta 200:

```
{
  "status": "200 OK",
  "reservations": [
    {
      "id_reservation": 10,
      "date": "2026-05-20",
      "time_start": "08:00:00",
      "time_end": "10:00:00",
      "status": "active",
      "laptop": {
        "id_laptop": 7, "status": "unavailable",
        "model": { "id_model": 2, "brand": "Lenovo", ... },
        "locker": { "id_locker": 1, "name_locker": "Locker A", ... }
      },
      "user": { "id_user": 42, "name": "Mario", "surname": "Rossi", ... }
    }
  ]
}
```

Struttura aggregazione del result set (posizioni per counter):

```
// counter 0-4: id_reservation, date, time_start, time_end, status ->
reservation root

// counter 5-10: id_user, name, surname, email, password, role ->
reservation['user']

// counter 11-12: id_laptop, status -
> reservation['laptop']

// counter 13-18: colonne models -
> reservation['laptop']['model']

// counter 19-21: colonne lockers -
> reservation['laptop']['locker']
```

6.17 POST /api/v1/reservations — Crea Prenotazione

Crea una o più prenotazioni per una fascia oraria, assegnando uno o più laptop. Richiede autenticazione. L'id_user è ricavato dal JWT.

Body:

Campo	Tipo	Obbligatorio	Descrizione
date	string	Sì	Data futura (>= oggi) in formato YYYY-MM-DD
time-start	string	Sì	Orario inizio HH:MM:SS
time-end	string	Sì	Orario fine HH:MM:SS (deve essere > time-start)
id-laptop[]	integer[]	Sì	Array di uno o più ID laptop (devono essere tutti 'available')

Logica di business critica:

- Verifica che la data sia >= oggi
- Valida il formato HH:MM:SS di time-start e time-end
- Verifica che time-end > time-start
- Controlla che ogni laptop richiesto sia status='available'
- Genera un id_reservation unico con MAX(id_reservation)+1
- Inserisce una riga in reservations per ogni laptop
- Aggiorna immediatamente lo status di ogni laptop a 'unavailable'

Dettaglio codice — inserimento multi-laptop con id condiviso:

```
$result = mysqli_query($conn, 'SELECT COALESCE(MAX(id_reservation),0)+1
AS next_id FROM reservations');
$nextId = mysqli_fetch_assoc($result) ['next_id'];

foreach ($laptopIds as $idLaptop) {
    // Inserisce riga con stesso $nextId per ogni laptop
    mysqli_stmt_execute($stmtR);
    // Marca il laptop come unavailable
    mysqli_stmt_execute($stmtL);
}
```

Risposta 201 Created:

```
{ "status": "201 Created", "id_reservation": 11 }
```

Risposta 409 Conflict — laptop non disponibile:

```
{ "status": "409 Conflict", "error": "I seguenti laptop non sono  
disponibili: 7, 12" }
```

Risposta 400 Bad Request — validazione fallita (data passata, orari errati, array laptop vuoto).

6.18 DELETE /api/v1/reservations — Cancella Prenotazione

Cancella una prenotazione. Un utente normale può cancellare solo le proprie prenotazioni; l'admin può cancellarne qualsiasi.

Body:

Campo	Tipo	Obbligatorio	Descrizione
id-reservation	integer	Sì	ID della prenotazione da eliminare

Dettaglio controllo autorizzazione:

```
$query = 'SELECT id_user FROM reservations WHERE id_reservation = ?';  
// ...  
foreach ($rows as $row) {  
    if ($token_decoded['payload']['admin'] || $row['id_user'] ==  
$token_decoded['payload']['id-user']) {  
        break; // autorizzato  
    } else {  
        // 401 Unauthorized  
    }  
}
```

Risposta 200 OK senza payload aggiuntivo.

7. Meccanismo di Filtraggio Dinamico

I GET endpoint (laptops, lockers, models, users, reservations) condividono un pattern di filtraggio dinamico basato sui query parameter. Il codice esegue prima una SELECT di un singolo record per ricavare i nomi delle colonne, poi costruisce dinamicamente la WHERE clause:

```
// 1. Query di 'schema discovery' per ottenere nomi colonne
$result = mysqli_query($conn, 'SELECT * FROM lockers WHERE id_locker = 1');
$row     = mysqli_fetch_assoc($result);

// 2. Per ogni colonna, controlla se esiste un query param corrispondente
//      (con underscore -> trattino per conformità REST)
foreach ($row as $attribute => $value) {
    $attr_dash = str_replace('_', '-', $attribute);
    if (isset($_GET[$attr_dash])) {
        $params_type    .= 's'; // o 'i' se intero
        $params_value[] = $_GET[$attr_dash];
        $params         .= $attribute . ' = ? AND ';
    }
}

// 3. Esecuzione con prepared statement parametrizzato
$stmt = mysqli_prepare($conn, $query . $params);
mysqli_stmt_bind_param($stmt, $params_type, ...$refs);
```

Questo approccio permette di filtrare per qualsiasi attributo della risorsa senza dover scrivere endpoint dedicati. Le colonne ambigue (presenti in più tabelle di una JOIN) vengono qualificate con l'alias della tabella tramite la funzione `qualify_column()`.

8. Codici di Risposta HTTP

Codice	Stato	Quando viene restituito
200	200 OK	Richiesta riuscita (GET, PATCH, DELETE)
201	201 Created	Risorsa creata (POST su laptops, lockers, models, reservations, users)
400	400 Bad Request	Parametri mancanti o invalidi (validazione prenotazione)
401	401 Unauthorized	Credenziali errate, utente non verificato, o non admin su endpoint admin
404	404 Not Found	Nessun record trovato o risorsa URI inesistente
405	405 Method Not Allowed	Metodo HTTP non supportato per quella risorsa
406	406 Not Acceptable	OTP errato/scaduto o utente già esistente
409	409 Conflict	Laptop non disponibile durante la prenotazione
498	498 Invalid Token	Token JWT assente, malformato, scaduto o firma non valida

9. Flussi d'Uso Tipici

9.1 Registrazione e Verifica

1. POST /api/v1/users — invia name, surname, email (@gmail.com), password
2. La risposta contiene un JWT non-verified e invia un OTP via email
3. POST /api/v1/otp — invia otp con il JWT ricevuto nell'header token
4. L'account è ora verificato; il JWT è scaduto (5 min). Eseguire il login.

9.2 Prenotazione di Laptop

1. POST /api/v1/login — ottieni il JWT (24h)
2. GET /api/v1/laptops?status=available — lista i laptop liberi
3. POST /api/v1/reservations — invia date, time-start, time-end, id-laptop[]
4. L'API verifica disponibilità, inserisce la prenotazione e marca i laptop unavailable
5. Gli eventi DB schedulati aggiornano automaticamente lo status al termine della fascia oraria

9.3 Annullamento Prenotazione

1. GET /api/v1/reservations?id-user=<proprio_id> — trova l'id_reservation
2. DELETE /api/v1/reservations — invia id-reservation nel body
3. L'API controlla che il chiamante sia il proprietario (o admin)